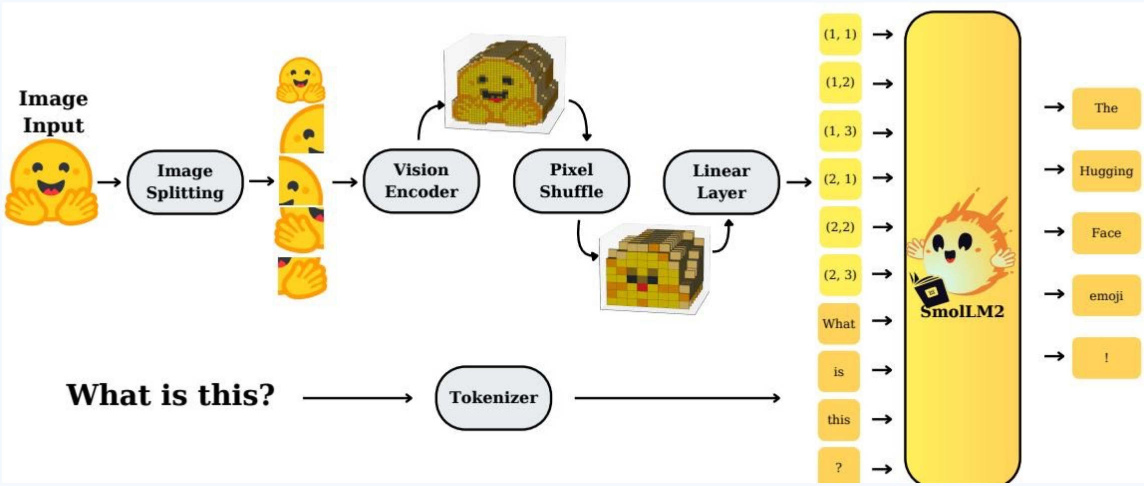
TECHNICAL PAPER BRIEF

SmolVLM:
Redefining Small and Efficie
Multimodal Models

Compact VLMs for resource-efficient inference on mobile and devices

Hugging Face & Stanford University



Core pipeline: image splitting → SigLIP encoder → pixel shuffle → projection → SmolLM2

0.8GB	16×	16K	59.8%
GPU RAM for 256M at batch=1	visual token reduction at r=4	context for 2.2B variant	average benchmark score for 2.2B

PROBLEM FRAMING

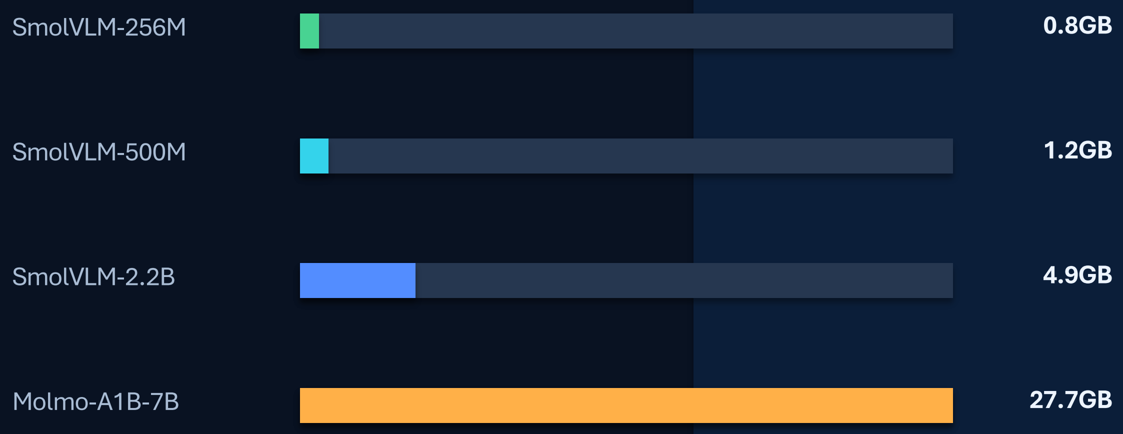
Large VLMs Are Powerful but Impractical for Edge Deployment

Edge constraint: memory and compute—not just accuracy—determine deployability.

- State-of-the-art VLMs require massive compute and memory, limiting mobile and low-power edge use.
- Molmo-A1B-7B requires 27.7GB RAM in the benchmark comparison.

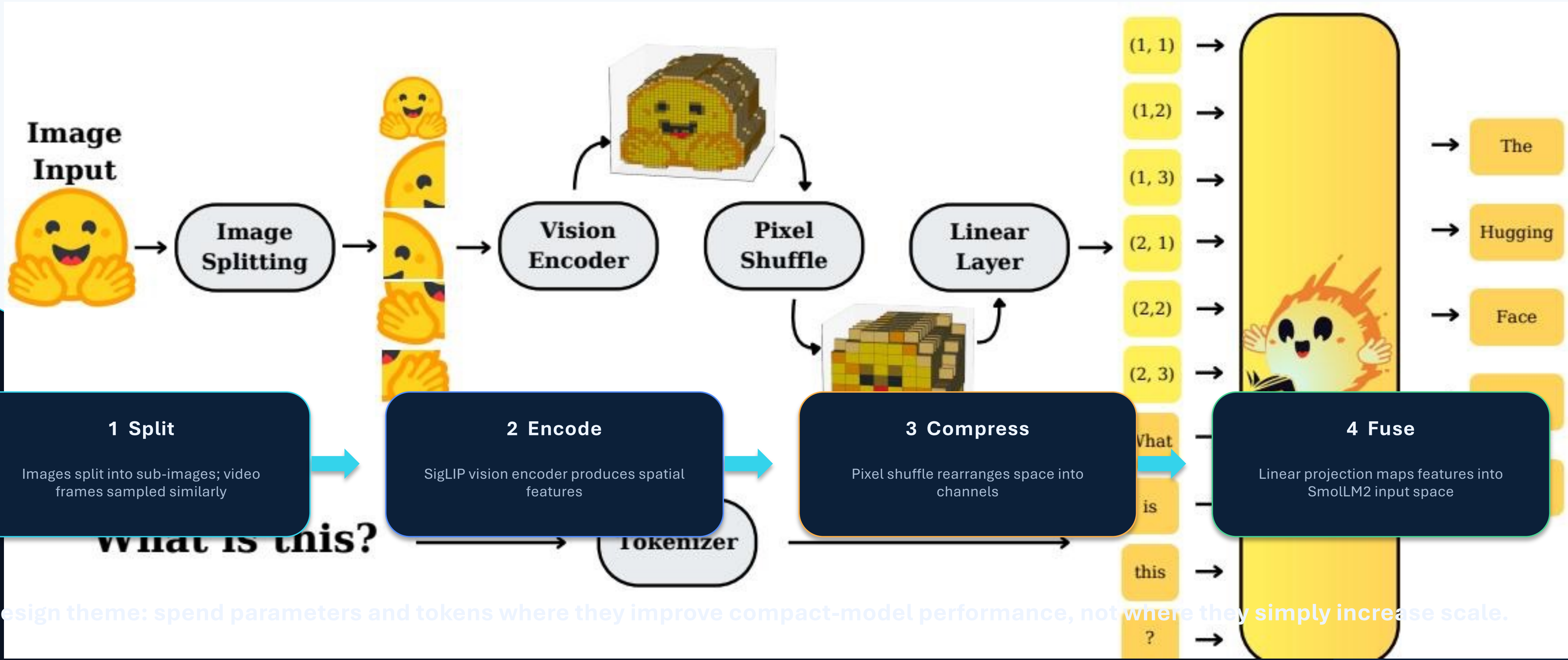
- SmolVLM targets competitive multimodal performance under dramatically lower RAM footprints.
- The 256M model runs inference with less than 1GB GPU RAM.

RAM usage at batch=1



Sub-1GB inference turns VLM deployment into a smartphone/edge-device systems problem—not a datacenter-only task.

SmolVLM Architecture: Image Splitting, Pixel Shuffle, and SmolLM2 Backbone



1 Split

Images split into sub-images; video frames sampled similarly

2 Encode

SigLIP vision encoder produces spatial features

3 Compress

Pixel shuffle rearranges space into channels

4 Fuse

Linear projection maps features into SmolLM2 input space

Design theme: spend parameters and tokens where they improve compact-model performance, not where they simply increase scale.

FINDING 1 — COMPUTE ALLOCATION

Smaller Vision Encoders Outperform Larger Ones in Compact VLMs

For a 135M language model, the smaller 93M SigLIP-B/16 encoder outperforms the 428M SigLIP-SO400M.

- Compact VLMs benefit from balanced encoder–LM parameter allocation.
- The larger encoder only justifies its 10% parameter overhead at 1.7B LM scale.
- Implication: vision capacity can crowd out language capacity in small VLMs.

Encoder comparison with the same 135M LM

SigLIP-B/16 — 93M

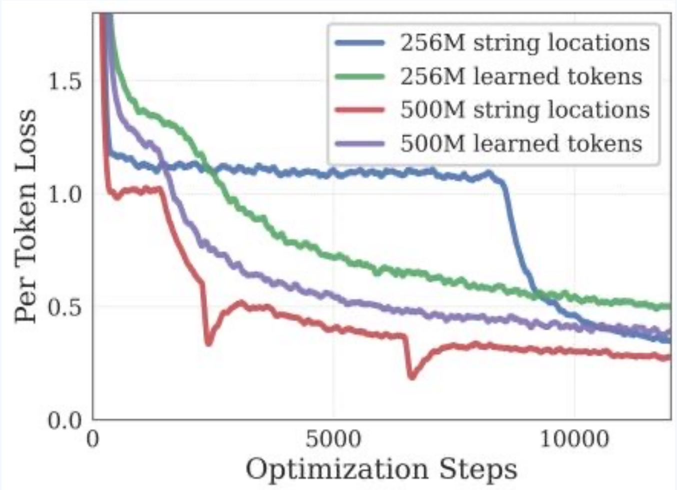
outperforms

SigLIP-SO400M — 428M

larger, lower for smallest LM

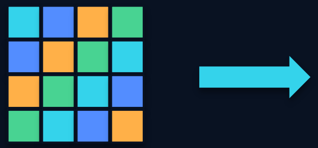
Design rule: match encoder size to LM scale; avoid over-investing in vision at the smallest language scales.

Aggressive Pixel Shuffle (r=4) Cuts Visual Tokens 16× for Small Models



Why it matters: visual tokens dominate attention cost when resolutions and context lengths grow.

Pixel shuffle rearranges spatial features into channels: tokens shrink by r^2.



1 token

16 channels

16×

token reduction at r=4

Shuffle factor preference by model scale

Small VLMs	r=4	eases attention overhead; improves long-context modeling
Larger VLMs	r=2	less aggressive compression preferred

FINDING 2 — CONTEXT LENGTH SCALING

Extending Context from 2K to 16K Tokens Unlocks Higher Image Resolution

Performance improves significantly when context extends from 2K to 16K tokens.

2K
original context

→

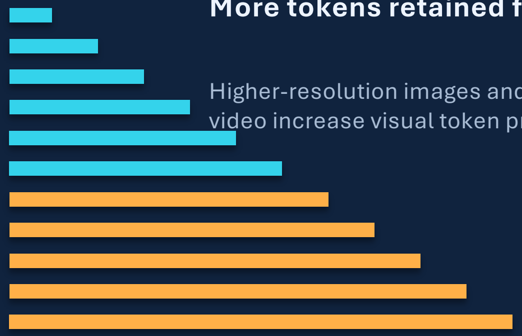
8K
256M / 500M variants

→

16K
2.2B variant

- RoPE base increased from 10K to 273K.
- Fine-tuning mixes long-context and short-context data.
- Longer context allows higher-resolution visual evidence to remain accessible.

Context extension strategy



More tokens retained for visual detail

Higher-resolution images and multi-frame video increase visual token pressure.

RoPE → 10K → 273K

Mixed long/short-context fine-tuning stabilizes the extension.

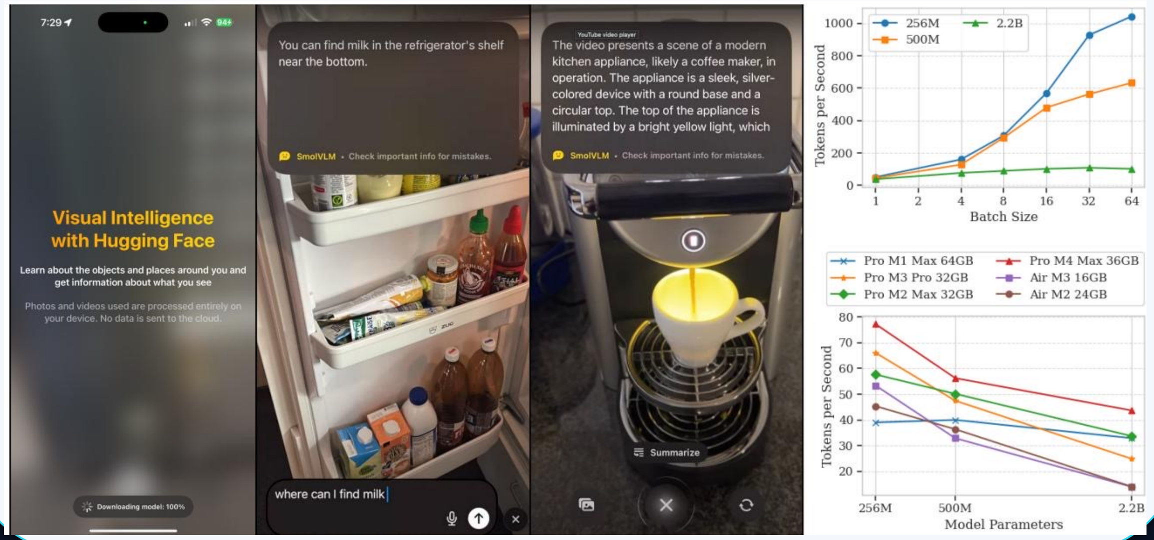
SmolVLM Achieves Competitive Performance at Sub-1GB to 4.9GB RAM

44.0%	51.0%	59.8%	0.8GB	4.9GB	27.7GB
256M average	500M average	2.2B average	256M RAM	2.2B RAM	Molmo-A1B-7B RAM

256M surpasses the 300× larger Idefics-80B.

Capability	Benchmark	256M	500M	2.2B	Best Efficient OS
Single-Image	OCRBench	52.6%	61.0%	**72.9%**	54.7% Molmo-A1B-7B
Single-Image	AI2D	46.4%	59.2%	**70.0%**	71.0% Molmo-A1B-7B
Single-Image	ChartQA	55.6%	62.8%	**68.7%**	48.0% Molmo-A1B-7B
Single-Image	TextVQA	50.2%	60.2%	**73.0%**	61.5% Molmo-A1B-7B
Single-Image	DocVQA	58.3%	70.5%	**80.0%**	77.7% Molmo-A1B-7B
Single-Image	ScienceQA	73.8%	80.0%	**89.6%**	87.5% Molmo-A1B-7B
Multi-task	MMMU	29.0%	33.7%	**42.0%**	33.9% Molmo-A1B-7B
Multi-task	MathVista	35.9%	40.1%	**51.5%**	37.6% Molmo-A1B-7B
Multi-task	MMStar	34.6%	38.3%	**46.0%**	43.1% Molmo-A1B-7B
Video	Video-MME	33.7%	42.2%	**52.1%**	45.0% InternVL2-2B
Video	MLVU	40.6%	47.3%	**55.2%**	48.2% InternVL2-2B
Average	**All Benchmarks**	**44.0%**	**51.0%**	**59.8%**	—
RAM	**Batch=1**	**0.8GB**	**1.2GB**	**4.9GB**	**27.7GB Molmo-A1B-7B**

Real-Time Inference on iPhones: Up to 80 Tokens/Second on Apple Silicon



HuggingSnap processes photos and videos locally with no cloud dependency.

Reported throughput highlights

Pro M1 Max 64GB
256M model
up to ~80 tok/s

MacBook Air M3 16GB
256M model
~55 tok/s

High-end devices
batch=64, 256M
nearly 1000 tok/s

- Throughput scales roughly linearly with batch size for the smallest model.
- Local photo/video processing aligns model design with privacy and latency constraints.

ACTIONABLE PRINCIPLES

Key Takeaways: Design Principles for Efficient Multimodal Models

Balance encoder–LM compute

Smaller vision encoders can outperform larger ones when the LM is compact.

Extend context length

Move beyond 2K; SmolVLM uses 8K for smaller variants and 16K for 2.2B.

Compress visual tokens aggressively

For small VLMs, r=4 pixel shuffle reduces visual tokens by 16×.

Optimize for deployable memory

Sub-1GB inference is achievable: SmolVLM-256M uses 0.8GB RAM.

Open the full stack

Weights, datasets, code, and the HuggingSnap mobile app are fully open-sourced.

Implication: efficient multimodal models are a co-design problem across architecture, sequence length, tokenization, and deployment constraints.